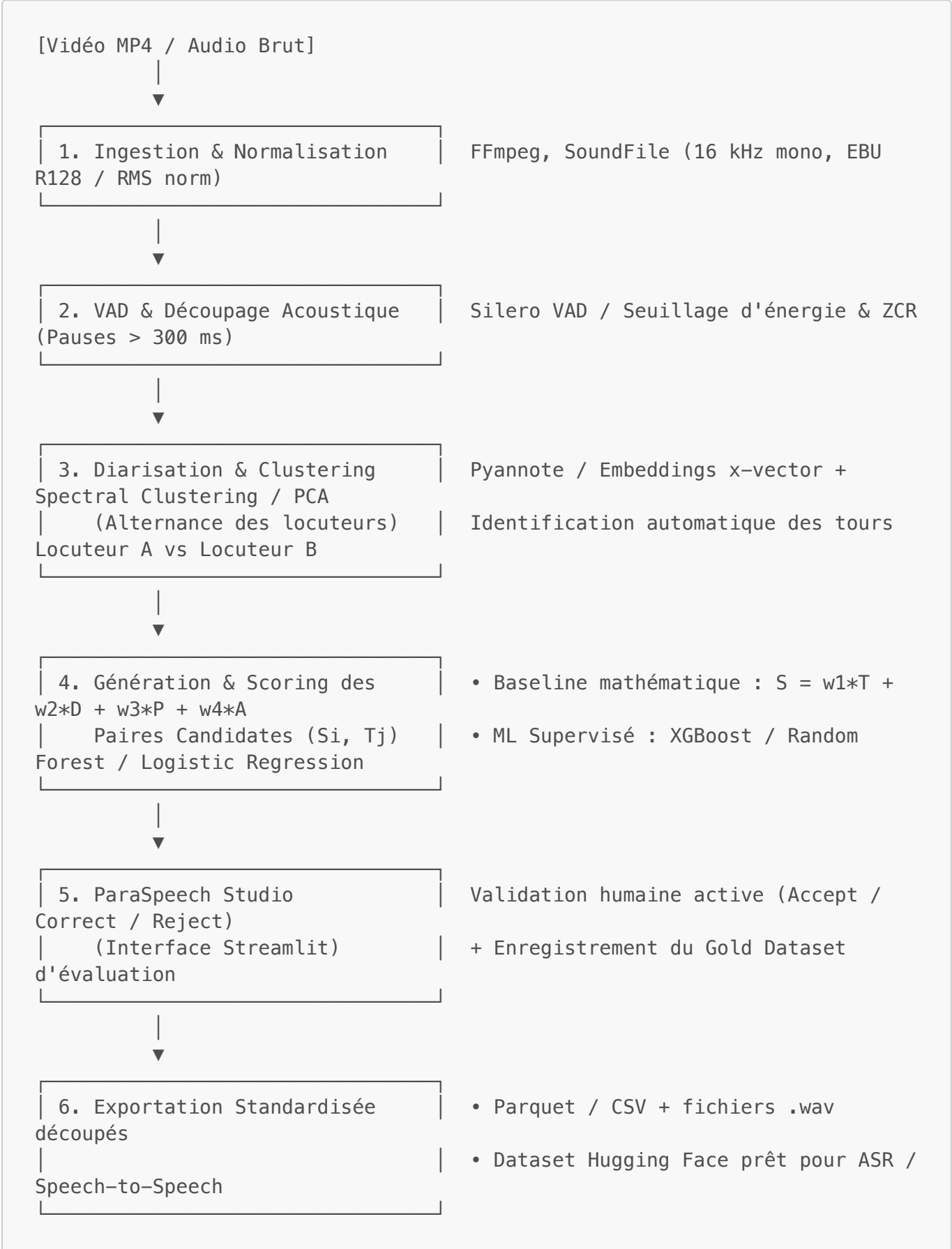
Architecture globale recommandée : Pipeline *Audio-to-Aligned-Corpus*

Le pipeline complet, calibré pour faire la passerelle entre l'ingénierie du signal classique et les architectures de pointe, s'articule en **6 modules séquentiels** :



Détail des étapes et fondements mathématiques

1. Ingestion, Conditionnement & Physique du Signal

- **Échantillonnage :** Conversion systématique en **16 000 Hz, mono (PCM 16-bit)**. Selon le théorème de Shannon-Nyquist, une fréquence d'échantillonnage de $f_s = 16\ \text{kHz}$ permet de restituer fidèlement la bande passante jusqu'à la fréquence de Nyquist $f_N = 8\ \text{kHz}$, ce qui englobe largement l'ensemble des formants de la voix humaine ($[300\ \text{Hz}, 3\ 500\ \text{Hz}]$ pour la parole fondamentale, harmoniques jusqu'à $7\ \text{kHz}$).
- **Normalisation d'intensité :**

  $$\tilde{x}[n] = \frac{x[n]}{\sqrt{\frac{1}{N}\sum_{i=1}^{N} x[i]^2}} \cdot \sigma_{\text{target}}$$

  Garantit que les seuils d'énergie et de détection de silence restent invariants face aux écarts de gain des microphones.

2. Détection d'Activité Vocale (VAD) & Segmentation

- **Signal classique :** Fenêtrage court (ex. Hamming de 25 ms, pas de 10 ms), calcul de la puissance moyenne par trame :

  $$E_m = \frac{1}{W}\sum_{k=0}^{W-1} x[m \cdot H + k]^2$$

  et du Taux de Passage par Zéro (*Zero Crossing Rate*, ZCR) pour discriminer fricatives, voyelles et bruits de fond.
- **Approche moderne :** Réseau récurrent léger (*Silero VAD*) pour filtrer les réverbérations d'église ou les bruits d'ambiance de salle.

3. Séparation des Locuteurs (Diarisation / Alternance Source-Cible)

Dans une interprétation consécutive, la structure sous-jacente est une **chaîne de Markov à alternance binaire quasi-stricte** :

$$\text{Locuteur } A \xrightarrow{\text{pause}} \text{Interprète } B \xrightarrow{\text{pause}} \text{Locuteur } A \ldots$$

- **Extraction d'empreintes :** Extraction d'un vecteur d'embedding d-dimensionnel $e_i \in \mathbb{R}^d$ par segment (x-vector / ECAPA-TDNN).
- **Partitionnement non-supervisé :** Clustering spectral ou K-Means contraint avec $k = 2$ locuteurs, suivi d'une projection PCA/t-SNE pour valider la séparabilité acoustique des deux voix.

4. Alignement et Scoring de Correspondance

Pour chaque segment source S_i et chaque segment interprété candidat T_j :

- **Baseline Mathématique Explicable :**

  $$\text{Score}(S_i, T_j) = w_1 T(i,j) + w_2 D(i,j) + w_3 P(i,j) + w_4 A(i,j)$$

  - $T(i,j) = \exp\left(-\frac{\Delta t_{ij}^2}{2\sigma_t^2}\right)$: pénalisation de la distance temporelle (proximité temporelle immédiate).
  - $D(i,j) = \exp\left(-\frac{|\ln(\text{dur}(T_j)/\text{dur}(S_i)) - \mu_r|^2}{2\sigma_r^2}\right)$: cohérence des ratios de durée d'élocution entre les deux langues.
  - $P(i,j) = \mathbb{I}(t_{\text{start}}(T_j) > t_{\text{end}}(S_i))$: causalité temporelle stricte.
  - $A(i,j)$: compatibilité de non-identité acoustique (l'interprète ne doit pas avoir la même signature vocale que l'orateur principal).
- **Modèle ML Supervisé :** Classification binaire ($Y \in \{0,1\}$) entraînée sur les features géométriques, temporelles et d'énergie via Scikit-Learn (Random Forest / Logistic Regression calibrée).

Voici les paramètres clés enregistrés pour notre travail :

- **Environnement :** Google Colab avec GPU NVIDIA CUDA.
- **Défi acoustique :** Audio mixé avec bruits ambiants, écho/musique de fond et deux micros distincts (ce qui est un avantage pour la séparation des timbres vocaux, mais demande un filtrage du bruit de fond).
- **Équipe :** Projet mené avec vos coéquipiers (Ghilth, Richnel, Wilfried).

Feuille de route immédiate : Phase 0 (Kick-off & Signal de base)

Conformément au programme de l'AMA, nous n'allons pas brûler les étapes avec des modèles lourds dès aujourd'hui. Nous allons d'abord :

1. **Choisir le couple de langues** et trouver une première vidéo pilote. lien
2. **Initialiser l'arborescence du projet** sur votre Google Drive / Colab.
3. **Extraire et formater l'audio brut** via FFmpeg en signal standard de laboratoire ($16\ \text{kHz}$, mono).
4. **Visualiser et mesurer mathématiquement le signal** (amplitude, RMS, durée).

Étape 1 : Le choix du couple de langues et de la première vidéo

Pour maximiser l'impact de **ParaSpeech Africa** (notamment au regard des initiatives citées dans votre cadrage comme *JaimeMaLangue* et la stratégie nationale d'IA au Bénin) :

- Le couple le plus documenté et accessible en vidéo consécutive en Afrique de l'Ouest est :
  - **Français** $\longleftrightarrow$ **Fongbe** (très fréquent dans les cultes, discours officiels et conférences au Bénin).
  - ou **Français** $\longleftrightarrow$ **Yoruba** (Bénin / Nigéria).

Action requise de votre part :

Trouvez **une seule vidéo YouTube** (5 à 15 minutes suffisent largement pour le pilote) qui respecte ces 3 critères :

1. L'orateur parle en français pendant 5 à 15 secondes.
2. L'interprète traduit oralement dans la langue locale immédiatement après.

3. Les deux personnes ne parlent pas en même temps (interprétation consécutive pure, pas simultanée).

(Exemples de recherche sur YouTube : "culte bilingue français fon", "prédication traduction fon", "discours bilingue yoruba français").

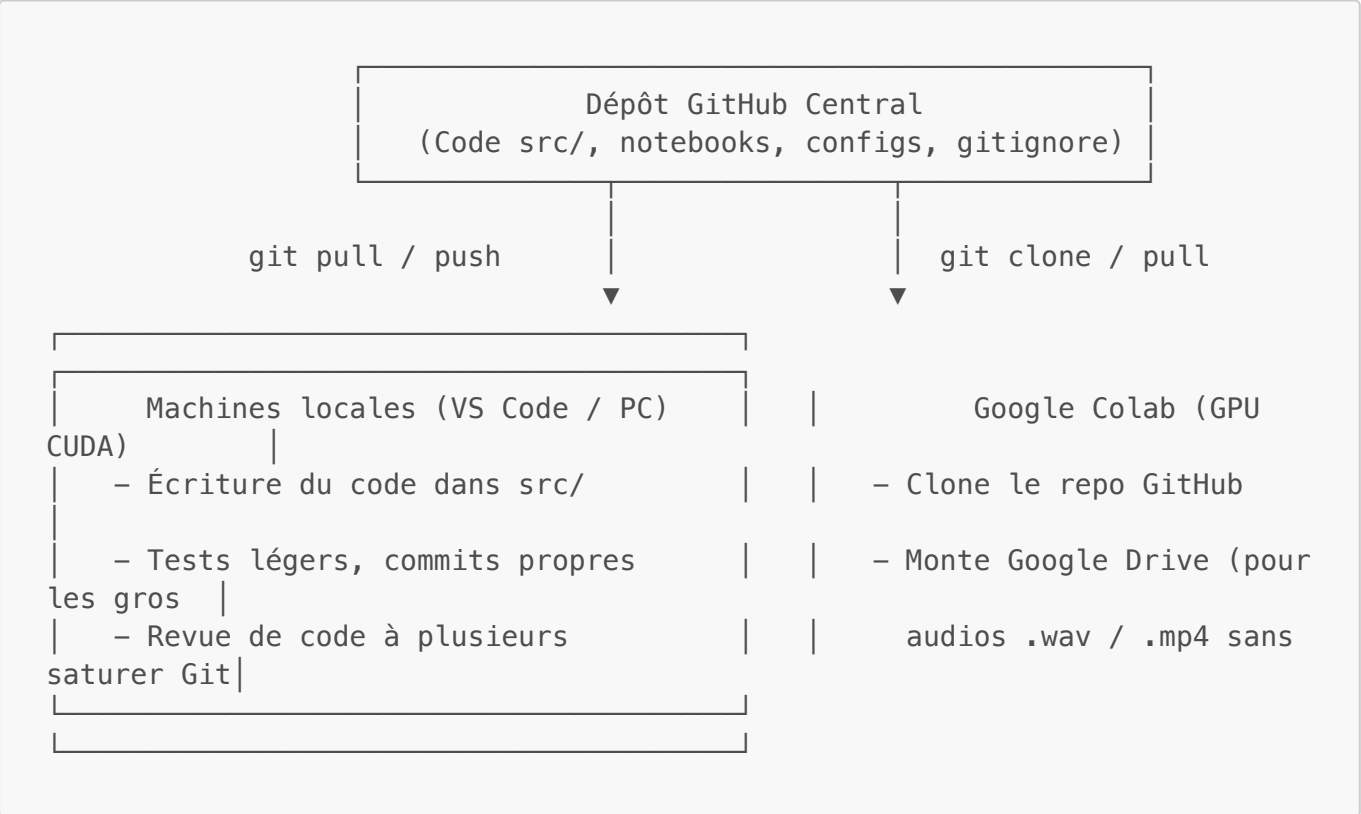
Tu as **parfaitement raison**. Ton intuition est exactement celle d'un ingénieur en production : **Google Drive seul n'est pas fait pour collaborer à 4 sur du code.**

Si vous travaillez tous les 4 directement dans un dossier partagé Google Drive, vous allez rapidement rencontrer des problèmes majeurs :

1. **Écrasement de fichiers** : deux personnes éditent le même fichier en même temps et l'une efface le travail de l'autre.
2. **Aucun historique** : impossible de savoir qui a fait quoi, ni de revenir en arrière si un bug est introduit.
3. **Pénalité académique** : le document de l'AMA indique explicitement : *"Livrable 1 : Repository GitHub propre, versionné, documenté et reproductible"* (noté à **10%**).

L'Architecture collaborative idéale : "GitHub pour le code, Colab pour le GPU, Drive pour l'audio"

Voici le flux de travail standard recommandé dans l'industrie et pour votre équipe (Ghilth, Richnel, Wilfried et toi) :



Comment mettre cela en place immédiatement

1. L'un de vous (l'Admin) crée le dépôt sur GitHub

- Nom du dépôt : `paraspeech-africa`
- Visibilité : Privé ou Public (selon les consignes de vos tuteurs AMA).
- Ajoutez les 3 autres comme **Collaborators** dans les paramètres du repo (*Settings* → *Collaborators*).

2. Définir impérativement le `.gitignore` (Règle d'or en Audio/ML)

Ne committez **jamais** de fichiers audio volumineux (`.mp4`, `.wav`, `.mp3`) sur GitHub (sinon le dépôt dépassera la limite de 100 Mo et bloquera).

Le fichier `.gitignore` à la racine doit contenir :

```
# Données lourdes (stockées sur Drive, pas sur GitHub)
data/raw/*
data/processed/*
!data/raw/.gitkeep
!data/processed/.gitkeep

# Checkpoints et gros modèles
*.pt
*.bin
*.onnx

# Python et Jupyter
__pycache__/
*.pyc
.ipynb_checkpoints/
.env
```

Note : Vous pouvez en revanche versionner les métadonnées légères dans `data/annotations/` (fichiers `.csv` ou `.json`).

---

3. Comment utiliser GitHub depuis Google Colab

Quand vous avez besoin de la puissance du GPU pour faire tourner un traitement lourd dans Colab, voici le modèle de notebook à exécuter en première cellule :

```
# 1. Monter Google Drive pour accéder aux gros fichiers audios
from google.colab import drive
drive.mount('/content/drive')

# 2. Cloner le dépôt GitHub (remplacez avec vos accès)
# Astuce : utilisez un GitHub Personal Access Token (PAT) pour cloner et
pusher facilement
%cd /content
!git clone https://github.com/VOTRE_ORGANISATION_OU_USER/paraspeech-
```

```
africa.git
%cd paraspeech-africa

# 3. Installer les dépendances du repo
!pip install -r requirements.txt -q
```

Prochaine étape :

Est-ce que le dépôt GitHub est déjà créé ou souhaites-tu que je te donne les commandes exactes (`git init`, `git remote add...`) pour initialiser le projet proprement depuis la machine locale de l'un d'entre vous ?